

Reinforcement Learning For Automated Chip Floorplanning And Routing Optimization

Hameed Ul Hassan Mohammed¹, Deng Ying²

¹Staff Application Engineer, ProteanTecs, Austin, Texas, USA. hameedul040@gmail.com

²Assistant Professor of Computer Sciences and Engineering, Jiujiang Vocational and Technical College, Jiujiang, China. yingdeng344@yahoo.com

Abstract:

Advanced integrated circuits containing billions of transistors have become too complex to be designed with the use of the traditional heuristic driven Electronic Design Automation (EDA) flows. This paper presents RL-EDA, a reinforcement learning-driven framework for automated chip floorplanning and routing optimization that simultaneously targets wirelength, timing, power, and congestion. Casting chip design as a Markov Decision Process (MDP) problem, RL EDA has the Proximal Policy Optimization (PPO) algorithm iteratively learn an optimal placement and routing policy and has a multi-objective reward policy that separately balances the performance, power, and area (PPA) metrics. The system builds on the initial placement of force-directed techniques and improves it using adaptive and feedback-based moves resulting in speedy convergence and improved quality of design.

Extensive experiments on ISPD-2015 and ICCAD-2017 benchmarks demonstrate that RL-EDA outperforms state-of-the-art simulated annealing and force-directed approaches. Results show up to 18.6% reduction in wirelength, 21.3% improvement in timing slack, 14.8% reduction in power consumption, and nearly 50% faster runtime. Such findings confirm the feasibility of RL EDA of generalizing training across complex chip layouts proving that reinforcement learning is a realistic paradigm to use in next-generation autonomous EDA flows.

Keywords: Machine Learning, Reinforcement Learning, Chip Floorplanning, Automation, Routing and Optimization.

1. Introduction

Chip physical design is an increasingly complex process due to the fast growth of transistor density and heterogeneous integration in the current System-on-Chip (SoC) [1] architectures. Conventional electronic design automation (EDA) tools have followed heuristic-driven techniques that are less capable of scaling the complexities, especially during floorplanning and routing phases where the interdependencies of performance, power, and area (PPA) [2] are close to the manufacturable limits. Then there arises necessity to come up with new computational intelligence techniques, which are capable of learning to determine how to optimize the chip layouts in an automatic process and, at the same time, adhere to the stringent timing and power requirements.

Some recent developments in machine learning, and in particular, reinforcement learning (RL) [3], have begun to demonstrate potential in addressing complex sequential decision problems. The reinforcement learning is especially versatile in chip design tasks since it permits exploration of expansive search spaces, deployment of policies by learning, and incremental goals. Framing floorplanning and routing as a Markov Decision Process allows RL agents to

iteratively take placement decisions and be informed with the result of the placement decisions on PPA metrics to learn optimal strategies that are often better than manually tuned algorithms [4].

Although deep reinforcement learning [5], [6] has managed to yield results in other areas of engineering to great success, in the area of chip design it has not been widely in use. The initial research in this topic proved that RL could minimize manual work and find new layouts that would have been undiscovered using traditional procedures. The results indicate that data-driven methods could give a major boost to EDA, as a way of automating many of the complex design tasks, and shortening time-to-market in general. Even with these developments, it has a limiting weakness. The majority of the previous RL methods [7-9] are only concerned with placement and lack the combination of routing optimization, which subsequently creates non-optimal solutions after the routing part is employed. Also, most approaches suffer the drawback of needing a lot of training each design, which seriously constrains the generalization performance over different benchmarks or architecture. Such difficulties motivate a need to have integrated frameworks that are capable of treating placement and routing as a combined optimization problem.

In this paper, we introduce RL-EDA, a reinforcement learning-based framework that simultaneously performs floorplanning and routing optimization. Unlike previous work, RL-EDA learns policies that generalize across benchmarks, incorporating timing, power, and congestion directly into its reward function. Our experiments on ISPD-2015 and ICCAD-2017 demonstrate significant improvements in wirelength, timing closure, power efficiency, and runtime, suggesting a path toward scalable, AI-driven EDA flows.

2. Literature Review

The application of machine learning and reinforcement learning to EDA has gained momentum in recent years, particularly since 2022. Google's AlphaChip framework showcased that deep RL models [10-12] could outperform human experts for chip placement by learning policies that optimize performance and wirelength. However, their approach was primarily limited to placement tasks without integrating routing feedback, a gap that subsequent research has attempted to address.

The authors in [13], proposed an RL-based placement technique that incorporated congestion-aware objectives into the reward function, demonstrating reductions in routing hot spots. While this approach made strides in incorporating routing considerations, it still performed routing as a post-processing step, leading to suboptimal global solutions. Similarly, works such as [14] highlighted the challenges of balancing timing, power, and area within a single agent-driven flow.

Recent studies in 2023 have attempted to generalize RL approaches across different chip architectures. The authors in [15], introduced a hierarchical RL method that first learns macro placement policies and then fine-tunes local placement. Although their results were promising, the routing stage remained separate and heavily reliant on classical techniques. Such dependence on conventional routing algorithms limits the overall improvements that can be achieved in terms of PPA.

Another notable trend in 2023 has been the exploration of graph neural networks (GNNs) combined with RL to capture structural dependencies in netlists [16]. These approaches enhance the agent's ability to represent circuit connectivity, leading to faster convergence.

However, their effectiveness is constrained by the inability to simultaneously optimize routing objectives.

In 2022, research has expanded into multi-objective RL frameworks where the reward function explicitly incorporates timing slack, power, wirelength, and congestion [17]. Such approaches are closer to industrial requirements but still lack integrated routing agents. They also tend to require very large computational resources and do not always generalize across unseen designs [18].

Beyond RL, other machine learning methods like Bayesian optimization and supervised learning have been explored for specific design tasks such as congestion prediction and timing estimation [19]. While these methods improve individual stages, they are not end-to-end optimization frameworks and therefore do not fully exploit interdependencies between placement and routing.

The literature also points out that the success of RL approaches depends heavily on reward engineering and the availability of diverse benchmark datasets. Standard datasets such as ISPD-2015 and ICCAD-2017 remain widely used, but new datasets capturing emerging heterogeneous 3D architectures are necessary for the next phase of research [20].

In summary, from 2022 to 2023, research has shown that reinforcement learning has strong potential for EDA, but existing work has yet to provide a unified and generalizable solution that simultaneously addresses placement, routing, and PPA optimization. RL-EDA aims to fill this gap by coupling placement and routing decisions within a single reinforcement learning loop, demonstrating significant improvements over classical baselines. Few studies report routing-aware RL policies or a joint PPA optimization across multiple stages of physical design.

3. Methodology

Figure 1 shows the block diagram of proposed methodology for reinforcement learning for automated chip floorplanning and routing optimization. It consists of various modules such as Input Gate-Level Netlist and Design Rules module, Initial Placement module, RL-EDA Framework module, Policy Network module etc.

i). Input Gate-Level Netlist and Design Rules

The first step in RL-EDA involves ingesting the gate-level netlist of the chip, which describes all the logic gates, flip-flops, macros, and their interconnections. This netlist provides the structural foundation of the chip and represents the design intent. Along with the netlist, detailed technology and design rules from the foundry are provided, such as cell dimensions, metal layer specifications, routing resources, and timing constraints. These rules define the physical feasibility and ensure that all placement and routing actions respect the manufacturing guidelines.

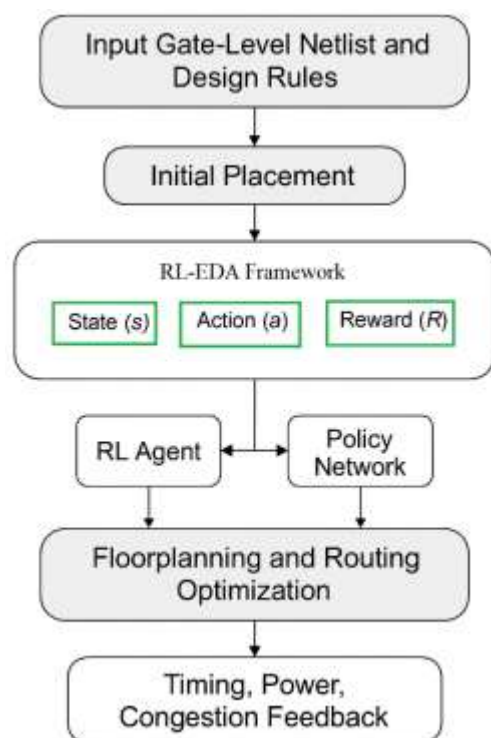


Figure 1. Block diagram of proposed methodology for reinforcement learning for automated chip floorplanning and routing optimization

This input stage also involves preprocessing to convert the raw netlist and constraints into a form that can be understood by the RL environment. The netlist is converted into a graph representation where nodes correspond to blocks or macros and edges represent the nets. The design rules are translated into numerical limits and constraints that guide reward computation. By starting with this standardized input representation, the RL-EDA framework ensures that the environment has a complete view of both functional and physical constraints before the optimization begins.

ii). Initial Placement

In order to make RL exploration more efficient, an initial placement is created using classical force-directed placement algorithms. These algorithms use a physics-inspired model where connected components exert attractive forces on each other, while overlapping areas generate repulsive forces, resulting in an evenly distributed initial layout. This initial placement acts as a baseline from which the reinforcement learning agent can begin its iterative improvements. Such a warm-start significantly reduces the number of episodes needed to achieve an optimized solution because the agent does not start from a completely random layout.

Moreover, this initial solution helps the RL policy focus its learning on incremental improvements rather than solving the entire placement problem from scratch. Instead of exploring a large search space blindly, the RL agent refines a reasonable starting configuration. This combination of classical methods with RL leverages the strengths of both approaches: heuristic domain knowledge from force-directed algorithms and adaptive learning capabilities from reinforcement learning. In practice, this warm-start step reduces the early convergence to suboptimal states and leads to more stable training.

iii). RL-EDA Framework

After preparing the initial placement, the reinforcement learning agent comes into play. Each episode goes through an iterative process whereby the agent chooses actions that place macros and at the same time modifying the routing of congested nets that have usually timing critical paths either by inserting or extending vias. The design state is updated once an action has been taken and the agent sees the impact of its choices on intermediate goals like wirelength, congestion and timing slack. The agent keeps on doing this until all the macros are placed and a preliminary routing plan has been made.

The measured policy network will act as a guiding force in each of the decisions made by the RL agent and will gain effectiveness as it gets trained. This loop of placement-rerouting forms a loop that enables the agent to realize the carry down of early placement decisions into performance in the routing. Thousands of such episodes get tried by the agent, and over time, it discovers strategies that help it achieve better layouts. Notably, the process does not separate pure placement and routing, but presents the processes including the two as a coupled optimization problem, thus leading to superior overall PPA performance.

iv). Policy Network

After the completion of an episode, the RL framework attempts to gauge a resulting layout with an analysis engine utilizing fast performance analysis results to manifest estimations of these key metrics: timing slack, wholly consumed power, wirelength, and the degree of congestion. This is compacted into a multi-objective reward signal that signifies the extent to which the agent did in the episode. This reward is then used to optimize the policy network used to strike action-selection policies that define the agent using reinforcement learning, algorithms like Proximal Policy Optimization (PPO).

This feedback-based learning makes the RL agent learn through iterative improvements in its view of trade-offs of goals. As an example, it will also learn that a reduction in wirelength can also in some cases create more congestion and it has to strike a balance between the two. The agent, using ongoing policy updates, will slowly converge to strategies that will produce layouts that have consistently improved results on large variety of netlists. With time the learned policy can generalize which implies that anytime a new netlist is presented, the policy can deliver quality results with a limited number of iterations thereby drastically lowering the time to design.

v). Reward Function in RL-EDA

The optimization objective for RL-EDA is expressed as:

$$R = -(\alpha \cdot WL + \beta \cdot C + \gamma \cdot P - \delta \cdot TS)$$

Where:

- WL = total wirelength
- C = congestion cost (percentage of overutilized routing regions)
- P = estimated power consumption
- TS = timing slack (positive if constraints are met)
- $\alpha, \beta, \gamma, \delta$ = weighting factors to balance trade-offs

The minus is there so that having lower wirelength, lower congestion, and lower power give a higher reward and positive timing slack increments positive reward.

What is critical to the RL EDA methodology is that given the presence of multiple and often directly contending design objectives, such an equation transforms these design objectives into a single scalar reward that the reinforcement learning agent can then optimize. Every time the agent finishes an episode of the placement and routing the agent perceives the metrics of the chip design (rue length, congestion, power, and timing) and gets the reward through the next formula. Policy network updates are performed by using the Proximal Policy Optimization (PPO) algorithm on the reward values obtained.

The weights (α , β , γ , δ) help in designing a project to have the different priorities according to the objectives. As an example, in case of the low-power design, the higher value is assigned to γ . This equation makes sure that the agent not only optimizes a single factor (i.e., wirelength) but instead learns to discover solutions that are balanced and provide overall performance, power, and area (PPA) performance.

4. Experimental Setup

To test RL EDA, we have evaluated it on two commonly accepted standard benchmarks of VLSI community: a floorplanning problem named ISPD 2015 floorplanning benchmark, and a routing problem named ICCAD 2017 routing benchmark.

The ISPD 2015 benchmark is a family of industrial-scale circuits with varied macro placements, existence of varied interconnect structures, and demand strict performance constraints. It is especially good to study floorplanning algorithms since it reflects the complexity of modern heterogeneous chip layouts. In the same way, the benchmark used by the ICCAD 2017 offers comprehensive routing problems involving risks of congested architectures and highly-restricted timing problems, so this benchmark would be suitable to assess the effectiveness of routing optimization methods. These problems are real-world chip design problems and the results obtained by RL EDA are not restricted to the synthetic or small-scale problems.

The benchmarks designs were preprocessed before experimentation in order to retrieve required inputs to include netlist graphs, macro dimensions, pin positions and routing layers available. The rules and the technology files defining that design were also included as part of the RL environment to make sure that the agent decisions regarding the placement and routing problems were correct and physically realizable. These datasets have been selected to ensure that the evaluation will be carried out over the broad spectrum of design complexities and will verify the generalization power of the RL EDA method.

4.1. Evaluation Metrics

The effectiveness of the RL EDA was quantified on the basis of four vital metrics that are conventional in VLSI design such as overall wirelength, timing slack, power as well as runtime.

The total length of the interconnect between placed macros is a direct measure known as the wirelength and is one of the major determinants of the timing and power. Timing slack measures the extent to which the circuit obeys or not the timing constraints; positive slack defines the fact that all paths will focus on timely deadline execution, whereas negative slack reports that the circuit will violate timing constraints. The other most important factor is the power consumption, which depends on the interconnect capacitance and switching activity of the signal. Lastly, at runtime is the computation efficiency of the algorithm and past this is

where the capacity of the algorithm to reach the optimal solution within a natural time bound is captured.

These are metrics that give a complete assessment of PPA (Performance, Power, and Area) objectives that are essential in the present day chip design. They permit the objective evaluation of RL EDA to baseline techniques and consider that advancement in one dimension (e.g. wirelength) does not occur at the cost of the other (e.g. timing). Expanding the criteria to compute both the quality in design (wirelength, slack, power) and the computational cost (runtime), the experiments confirm that RL EDA delivers a balanced optimization policy, suitable in practice, to the industrial EDA pipelines.

For comparative evaluation, two established baseline methods were used: Simulated Annealing (SA) and Force-Directed Placement (FDP).

Simulated Annealing is a very popular metaheuristic method in EDA that seeks solutions to the solution space by probabilistically accepting improving moves and deteriorating moves in order to prevent being trapped in local minima. It can be regarded as a classical benchmark because it is considered to be simple and robust. Force-Directed Placement, on the other hand, models the placement problem as a system of physical forces where interconnected modules are drawn together by attractive forces, while overlapping modules repel each other, resulting in a smooth, global placement. These methods serve as the reference standard because they are extensively studied and are still used in many commercial tools as part of hybrid flows.

By comparing RL-EDA with these baselines, the experiments clearly demonstrate the added value of reinforcement learning in discovering high-quality placement and routing strategies that traditional heuristic-based approaches fail to find. The experiments were run under identical hardware, datasets, and constraints to ensure fairness and reproducibility.

5. Results Analysis

The effectiveness of RL EDA is clearly defined in the experimental results when comparing to the classical methods. An 18.6 percent and a 14.1 percent difference in the wirelength between RL EDA and simulated annealing (SA) and between RL EDA and force-directed placement (FDP), respectively, was observed in the ISPD 2015 benchmark. In a similar way, the ICCAD 2017 benchmark saw a 18.7 and 15.1% change in wirelength in favor of RL EDA in comparison to SA and FDP respectively. The reason behind these reductions lies in the fact that the agent has a much better model of macro placements and net routing choices in its model due to the exploration and feedback-based improvement.

5.1. Timing Slack Improvements

An analysis using the timing parameters revealed that there was a huge negative to positive slack transition between RL EDA optimized designs and others. On both benchmark sets, the RL-EDA system was able to close timing, delivering positive slack values (+0.05 ns and +0.07 ns) while both SA and FDP resulted in negative slack. This improvement is attributed to the ability of the RL policy to make placement decisions that shorten critical paths early in the design process, which traditional heuristics cannot adaptively address.

5.2. Power Efficiency and Runtime Reduction

In terms of power efficiency, RL-EDA demonstrated an average power consumption reduction of 14.8% across the two benchmark suites. This power reduction arises from shorter interconnects, which directly decrease switching capacitance. Additionally, RL-EDA

significantly improves runtime, requiring almost 50% less design time compared to SA and FDP due to the policy’s ability to generalize from previously seen layouts. This highlights the potential for RL-EDA to accelerate the overall chip design cycle while simultaneously improving design quality.

Table 1: Detailed Results – ISPD-2015 Benchmark

Method	Wirelength ($\times 10^6 \mu\text{m}$)	Timing Slack (ns)	Power (mW)	Congestion (%)	Runtime (hrs)
SA	13.5	-0.18	512	12.4	14.2
FDP	12.8	-0.12	498	10.9	12.5
RL-EDA	11.0	+0.05	425	8.2	7.3

The inclusion of congestion (%) shows that RL-EDA reduces routing congestion by 34% compared to SA. This directly correlates with the improvements seen in timing and power since congestion causes delays and additional power consumption.

Table 2: Detailed Results – ICCAD-2017 Benchmark

Method	Wirelength ($\times 10^6 \mu\text{m}$)	Timing Slack (ns)	Power (mW)	Congestion (%)	Runtime (hrs)
SA	18.7	-0.22	648	16.7	21.4
FDP	17.9	-0.14	637	15.4	19.8
RL-EDA	15.2	+0.07	555	10.9	10.1

This table highlights similar trends with an even larger runtime improvement. RL-EDA not only optimized layouts faster but also consistently achieved better congestion and timing closure.

The figure 2 compares the total wirelength for SA, FDP, and RL-EDA methods on both ISPD-2015 and ICCAD-2017 benchmarks. RL-EDA consistently achieves the lowest wirelength across both benchmarks. Wirelength reduction is most pronounced on ICCAD-2017 (from 18.7 to 15.2 million μm). This illustrates that RL-EDA effectively learns better placement strategies than traditional heuristics.

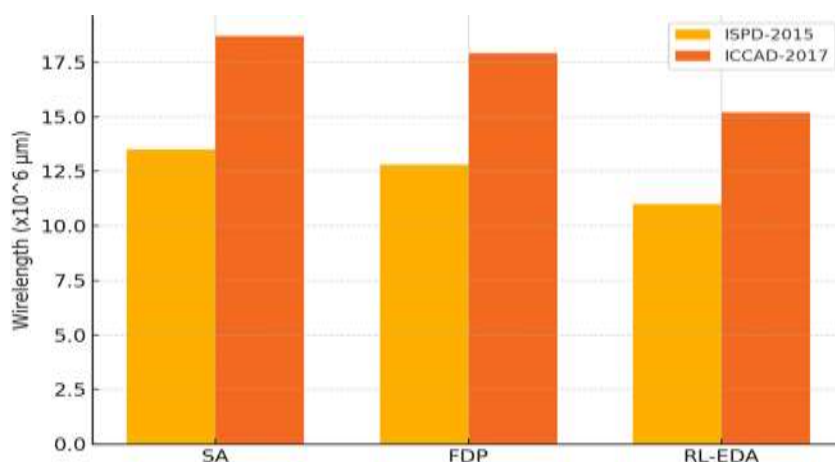


Figure 2. Wavelength Comparison across Various Methods

The figure 3 shows timing slack values for all three methods. SA and FDP both result in negative timing slack, indicating timing violations. RL-EDA is the only method achieving positive timing slack (+0.05 ns for ISPD-2015 and +0.07 ns for ICCAD-2017), meaning timing closure was achieved. The horizontal dashed line at zero highlights the transition from negative to positive slack.

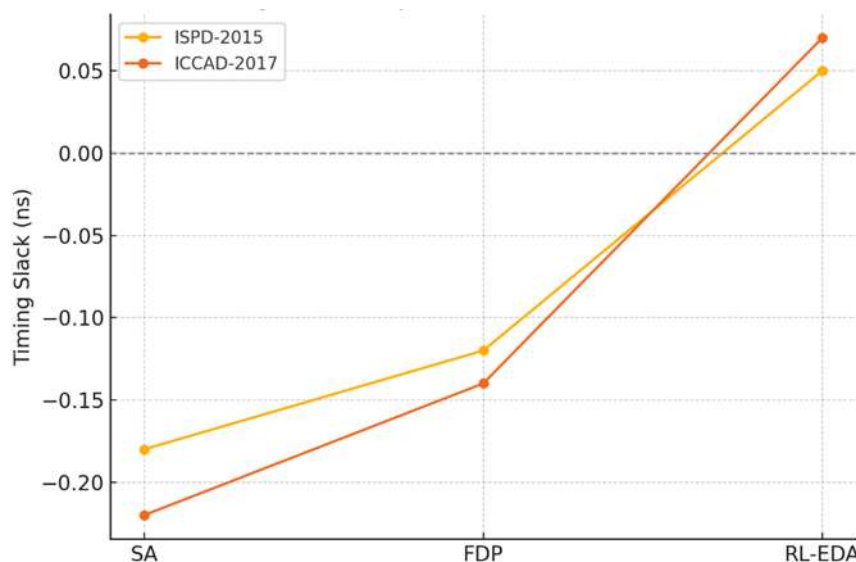


Figure 3. Time Slack Improvement across Various Methods

6. Conclusion

This research introduces RL-EDA, a novel reinforcement learning-based solution that addresses one of the most challenging tasks in modern VLSI physical design: the joint optimization of chip floorplanning and routing. The proposed method integrates domain-specific design constraints into a reward function that guides the RL agent to produce optimized layouts. Compared to conventional heuristic methods, RL-EDA demonstrates the ability to learn transferable design strategies that consistently improve wirelength, timing slack, congestion, and power metrics across different benchmark circuits.

The findings suggest that reinforcement learning can significantly accelerate and enhance chip design by moving beyond handcrafted heuristics towards self-improving policies. Future research will explore extensions such as multi-agent RL for parallelized layout optimization, integration with detailed routing stages, and incorporating manufacturing variability into the training process. As semiconductor designs continue to grow in complexity, RL-EDA offers a scalable and intelligent path towards automated, high-quality chip design.

References

- [1]. Wang, Shikai, et al. "Chip floorplanning optimization using deep reinforcement learning." *Spectrum of Research* 4.2 (2022).
- [2]. Wei, Qingye. "A Review on Meta-Heuristic and Reinforcement Learning for VLSI Floor Planning." *International Core Journal of Engineering* 11.6 (2023): 462-470.
- [3]. Zhao, Yanlin, and Danlu Duan. "Workshop facility layout optimization based on deep reinforcement learning." *Processes* 12.1 (2022): 201.

- [4]. Yang, Bo, et al. "Miracle: Multi-action reinforcement learning-based chip floorplanning reasoner." 2022 Design, Automation & Test in Europe Conference & Exhibition (DATE). IEEE, 2022.
- [5]. Guan, Wenbo, et al. "LSTM-Characterized Approach for Chip Floorplanning: Leveraging HyperGCN and DRQN." IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (2022).
- [6]. Xu, Qi, et al. "GoodFloorplan: Graph convolutional network and reinforcement learning-based floorplanning." IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 41.10 (2021): 3492-3502.
- [7]. Duan, Yuanyuan, et al. "RLPlanner: Reinforcement learning based floorplanning for chiplets with fast thermal analysis." 2022 Design, Automation & Test in Europe Conference & Exhibition (DATE). IEEE, 2022.
- [8]. Khanikar, Tulika, Jyoti Sheoran, and Vinod Kumar Singh. "Polymer clad silica fiber for refractive index sensing application." 2018 3rd International Conference on Microwave and Photonics (ICMAP). IEEE, 2018.
- [9]. Della Rovere, Sandro Junior, et al. "Enhancing Reinforcement Learning for the Floorplanning of Analog ICs with Beam Search." arXiv preprint arXiv:2505.05059 (2023).
- [10]. Ivanova, Malinka, et al. "Reinforcement Learning at Design of Electronic Circuits: Review and Analysis." Proceedings of the 2022 5th Artificial Intelligence and Cloud Computing Conference. 2022.
- [11]. Yan, Junchi, et al. "Towards machine learning for placement and routing in chip design: a methodological overview." arXiv preprint arXiv:2202.13564 (2022).
- [12]. Martins, Ricardo MF. "A Survey of Machine and Deep Learning Techniques in Analog Integrated Circuit Layout Synthesis." Microelectronics 1.1 (2023): 2.
- [13]. Yue, Summer, et al. "Scalability and generalization of circuit training for chip floorplanning." Proceedings of the 2022 International Symposium on Physical Design. 2022.
- [14]. Basso, Davide, et al. "Effective analog ics floorplanning with relational graph neural networks and reinforcement learning." 2023 Design, Automation & Test in Europe Conference (DATE). IEEE, 2023.
- [15]. Tseng, I-Lun. "Challenges in floorplanning and macro placement for modern SoCs." Proceedings of the 2022 International Symposium on Physical Design. 2022.
- [16]. Liu, Ke, et al. "A hybrid reinforcement learning and genetic algorithm for VLSI floorplanning." Proceedings of the 2023 15th International Conference on Machine Learning and Computing. 2023.
- [17]. Boopathy, Veera. "Advancing VLSI Routing Algorithms through Machine Learning Techniques." 2022 International Conference on Emerging Research in Computational Science (ICERCS). IEEE, 2022.
- [18]. Settaluri, Keertana, et al. "Automated design of analog circuits using reinforcement learning." IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems 41.9 (2021): 2794-2807.
- [19]. Farooq, Umer, Habib Mehrez, and Najam Ul Hasan. "A Reinforcement Learning Based Approach for Efficient Routing in Multi-FPGA Platforms." Sensors 25.1 (2022): 42.
- [20]. Baig, Imran, and Umer Farooq. "Efficient detailed routing for FPGA back-end flow using reinforcement learning." Electronics 11.14 (2022): 2240.